

Réponses aux Exercices Théoriques et Conceptuels

TPs 1 à 5 : IA Générative et LangChain

Compte Rendu

Table des matières

1	TP1 : Premiers pas avec la génération de texte	2
1.1	Exercices 2.1 & 2.2 : Compréhension de LangChain	2
1.2	Exercices 3.1 & 5.2 : Analyse des coûts (Usage Metadata)	2
1.3	Rendu attendu : Synthèse TP1	2
2	TP2 : Sorties structurées	2
2.1	Exercices 1.1 & 1.2 : Intérêt de la structuration	2
2.2	Exercice 2.1 : Questions ambiguës	3
2.3	Exercice 2.2 : Importance du typage strict	3
2.4	Exercice 3.2 : Robustesse contre les hallucinations	3
2.5	Exercice 4.1 : Justification de la note	3
2.6	Exercice 5.1 : Fonctionnement du RunnableLambda	3
2.7	Exercice 6.2 : Limites du raisonnement mathématique	3
3	TP3 : Systèmes RAG et bases vectorielles	3
3.1	Exercice 1.1 & 1.2 : Les concepts	3
3.2	Exercices 2.1 & 2.2 : Bag of Words	4
3.3	Exercices 3.2 & 4.1 : Transformers et Similarité	4
3.4	Exercice 5.2 : Réduction de dimension	4
3.5	Exercices 9.1 & 10.2 : Industrialisation	4
4	TP4 : Orchestration d’agents avec LangChain	4
4.1	Exercice 0.1 : Cas d’usage Multi-Agents	4
4.2	Exercice 2.2 : Avantages et limites du Mono-outil	5
4.3	Exercice 3.1 & 4.2 : Analyse et Coûts (ReAct)	5
4.4	Exercice 7.2 : Gestion des erreurs de parsing	5
4.5	Exercice 9.1 : Orchestration séquentielle	5
5	TP5 : Multimodalité et IA générative	5
5.1	Exercice 0.1 & 0.2 : Cas d’usage	5
5.2	Exercice 2.1 & 3.1 : Audio	5
5.3	Exercice 4.1 & 5.1 : Vision et Images	6

1 TP1 : Premiers pas avec la génération de texte

1.1 Exercices 2.1 & 2.2 : Compréhension de LangChain

Cas d'usage concret (2.1) : LangChain pourrait être utilisé pour créer un assistant de support client interne. Il interrogerait la base de données documentaire de l'entreprise (connaissances RH ou IT) via un système RAG, et formulerait des réponses précises aux employés, réduisant ainsi la charge du support de niveau 1.

Composant intéressant de la documentation (2.2) : L'utilisation de *LangGraph* pour créer des agents cycliques capables d'évaluer leurs propres réponses et de se corriger avant d'afficher le résultat final à l'utilisateur (workflow *Self-Reflective*).

1.2 Exercices 3.1 & 5.2 : Analyse des coûts (Usage Metadata)

L'estimation des coûts d'une API de LLM se base sur le comptage des tokens en entrée (*prompt*) et en sortie (*completion*). Les tarifs varient selon le modèle.

Requête	Type	Tokens In	Tokens Out	Coût estimé (€)
Capitale Albanie	Simple, courte	15	10	~ 0.0001
Blague pompier	Créative, formée	30	50	~ 0.0003
Explication modèle	Technique, longue	40	250	~ 0.0015

TABLE 1 – Estimation théorique des coûts d'inférence

1.3 Rendu attendu : Synthèse TP1

- **Compréhension :** LangChain agit comme une couche d'abstraction (un "glue code" intelligent) permettant de standardiser les appels à des modèles hétérogènes (Mistral, OpenAI) tout en chaînant facilement des étapes (prompts, parsers).
- **Coûts :** Les coûts augmentent drastiquement avec la longueur du contexte. L'optimisation des prompts est financièrement cruciale.
- **Applications futures :** Intégration de bases vectorielles pour donner de la mémoire au modèle, ou utilisation de sorties structurées pour communiquer avec des API classiques.

2 TP2 : Sorties structurées

2.1 Exercices 1.1 & 1.2 : Intérêt de la structuration

Scénarios métiers (1.1) : 1) Extraction d'entités nommées depuis des contrats juridiques (montants, dates, signataires) pour remplir automatiquement une base de données SQL. 2) Un système de triage de tickets de support où le modèle doit obligatoirement retourner une catégorie préexistante (ex : "Facturation", "Technique").

Problèmes des sorties libres (1.2) : Le parsing casse fréquemment si le modèle décide d'ajouter un préambule poli ("Voici le résultat : ...") ou s'il omet des guillemets dans un JSON généré manuellement.

2.2 Exercice 2.1 : Questions ambiguës

Pour une question ambiguë comme "Les tomates sont-elles des légumes ?", un simple booléen ne suffit pas. Le prompt devrait inclure une instruction stricte : *"En cas d'ambiguïté scientifique ou culturelle, réponds strictement selon la classification botanique (True/False)."*

2.3 Exercice 2.2 : Importance du typage strict

Obtenir un vrai type booléen Python (et non la chaîne de caractères "True") évite les opérations de conversion chronophages et empêche les failles d'exécution dans la logique conditionnelle (`if/else`) de l'application logicielle sous-jacente.

2.4 Exercice 3.2 : Robustesse contre les hallucinations

Avec un schéma structuré (Enum), si le modèle hallucine une action (ex : "Contacter le patron"), le validateur Pydantic intercepte l'erreur immédiatement. Avec du texte libre, le programme tenterait d'exécuter une action inexistante, provoquant un crash système.

2.5 Exercice 4.1 : Justification de la note

Le prompt système doit être enrichi : *"Tu dois obligatoirement justifier la note attribuée dans le champ 'commentaire' en citant un mot clé ou une expression du texte d'origine qui a motivé ton choix."*

2.6 Exercice 5.1 : Fonctionnement du RunnableLambda

Sans le `RunnableLambda`, la chaîne retourne l'objet Pydantic complet. Cela est souvent plus utile car on peut exploiter les métadonnées (ex : la langue source détectée par le modèle) pour faire des statistiques ou router la réponse.

2.7 Exercice 6.2 : Limites du raisonnement mathématique

Les LLM textuels "calculent" par probabilité des tokens et non par arithmétique. Ils échouent fréquemment sur des grands nombres, des équations non linéaires ou des simplifications de fractions complexes, nécessitant plutôt de leur fournir un outil externe (comme un interpréteur Python).

3 TP3 : Systèmes RAG et bases vectorielles

3.1 Exercice 1.1 & 1.2 : Les concepts

Applications (1.1) : 1) Détection de plagiat documentaire. 2) Chatbot de documentation d'entreprise (RAG). 3) Moteur de recommandation d'articles de presse (suggérer des articles au vecteur proche).

Composants RAG (1.2) :

- **Ingestion** : Découpage des textes et vectorisation.
- **Vector Store** : Base de données stockant les vecteurs pour une recherche rapide.
- **Retrieval** : Algorithme (ex : k-NN ou similarité cosinus) trouvant les fragments pertinents pour la question.
- **Generation** : Le LLM synthétise une réponse finale en utilisant uniquement ces fragments.

3.2 Exercices 2.1 & 2.2 : Bag of Words

Ajout d'un synonyme (2.1) : La matrice va créer une nouvelle colonne pour ce mot. Le BoW traite les synonymes comme des mots complètement orthogonaux, ignorant leur lien sémantique.

Limites (2.2) : 1) Incapacité à comprendre le contexte (ex : "vol" d'oiseau vs "vol" de banque). 2) Matrice très creuse et coûteuse en mémoire face à un grand vocabulaire.

3.3 Exercices 3.2 & 4.1 : Transformers et Similarité

Device CPU (3.2) : Sans GPU (CUDA/MPS), forcer le CPU évite que PyTorch ne crashe en cherchant un matériel non disponible. **Similarité Cosinus (4.1)** : Deux phrases opposées auront un score proche de 0 (vecteurs orthogonaux), ou légèrement négatif. Un score proche de 1 indique une forte similarité.

3.4 Exercice 5.2 : Réduction de dimension

Passer de 3072 à 256 dimensions réduit le coût de stockage et accélère la recherche (K-NN), mais entraîne une perte de nuance sémantique (baisse de la précision de rappel sur des requêtes complexes).

3.5 Exercices 9.1 & 10.2 : Industrialisation

Chunking (9.1) : Un `chunk_size` trop grand dilue l'information (le LLM perd le fil) ; trop petit, le contexte est tronqué. L'`overlap` empêche qu'une phrase importante soit coupée en deux entre deux chunks.

Vector Stores (10.2) : FAISS est excellent en local/RAM. Chroma ou Milvus sont nécessaires pour la mise en production : ils offrent la persistance sur disque, la gestion de requêtes concurrentes, et le filtrage avancé par métadonnées (ex : chercher par date *avant* de faire la recherche vectorielle).

4 TP4 : Orchestration d'agents avec LangChain

4.1 Exercice 0.1 : Cas d'usage Multi-Agents

Un système d'intelligence économique pourrait utiliser un "Agent Chercheur" pour scraper le web à la recherche de concurrents, et un "Agent Analyste" pour traiter ces données brutes et générer un rapport financier comparatif.

4.2 Exercice 2.2 : Avantages et limites du Mono-outil

- **Avantages** : Prédicibilité totale, consommation de tokens maîtrisée, débogage facile.
- **Limites** : Incapacité à croiser des informations de différentes sources ou à changer de stratégie si l'outil échoue.

4.3 Exercice 3.1 & 4.2 : Analyse et Coûts (ReAct)

Dans `intermediate_steps`, l'agent démontre une phase d'évaluation ("Thought") où il choisit son outil en fonction des descriptions fournies, puis ("Observation") lit le retour de l'outil avant de conclure.

Fournisseur	Qualité de raisonnement (ReAct)	Coût relatif d'inférence
Mistral Large	Très bonne (formalisme strict)	Moyen
OpenAI GPT-4o	Excellente (gestion d'erreurs robuste)	Plus élevé

TABLE 2 – Comparaison théorique Mistral vs OpenAI pour les agents

4.4 Exercice 7.2 : Gestion des erreurs de parsing

Il est pertinent d'activer `handle_parsing_errors=True` en production. Si le LLM ne respecte pas strictement le formalisme (ex : oublie le préfixe "Action :"), l'exécuteur intercepte l'erreur, la renvoie au LLM avec un message correctif, et lui donne une seconde chance au lieu de faire crasher l'application.

4.5 Exercice 9.1 : Orchestration séquentielle

Le contrôleur principal (Superviseur) reçoit la requête utilisateur. Il sollicite d'abord l'Agent 1 (Recherche) qui renvoie des données brutes. Le Superviseur transmet ensuite ces données à l'Agent 2 (Calcul/Analyse), récupère le résultat final, et le présente à l'utilisateur.

5 TP5 : Multimodalité et IA générative

5.1 Exercice 0.1 & 0.2 : Cas d'usage

- 1) **Support de maintenance industrielle** : Entrée (Image d'une pièce cassée + Audio "Quel est le problème?") → Sortie (Texte de diagnostic + Audio des instructions).
- 2) **Accessibilité Web** : Entrée (Image/Graphique) → Sortie (Audio d'audiodescription).
- 3) **Création de contenu ludique** : Entrée (Texte scénario) → Sortie (Images illustratives + Voix off audio).

5.2 Exercice 2.1 & 3.1 : Audio

Voix (2.1) : Le choix du timbre dépend de l'application (ex : *Alloy* est neutre, *Nova* est plus dynamique pour le storytelling).

Chunks (3.1) : Découper un audio long permet de contourner les limites de taille de l'API (ex : 25 MB). Un **chunk** de 2 à 5 minutes optimise le temps de transfert réseau tout en laissant assez de contexte au modèle Whisper pour la ponctuation.

5.3 Exercice 4.1 & 5.1 : Vision et Images

Lecture manuscrite (4.1) : Le modèle **gpt-4o** (complet) est généralement bien plus performant que **gpt-4o-mini** pour déchiffrer des écritures manuscrites complexes ou de mauvaise qualité.

Génération d'images (5.1) : Augmenter la résolution (ex : 1024x1792) et la qualité (*high*) permet d'obtenir de meilleurs détails (visages, textures), mais augmente de manière drastique le temps de génération (parfois x3) et le coût par appel API.